

# MSU Graduate Spatial Ecology Lab 5

Phoebe Zarnetske: [plz@msu.edu](mailto:plz@msu.edu); Lala Kounta: [kountala@msu.edu](mailto:kountala@msu.edu); Kelly Kapsar: [kapsarke@msu.edu](mailto:kapsarke@msu.edu)

Last rev. Sep, 2025; created Sep, 2020

## Lab 5: Variograms, Ripley's K

*RMarkdown version of this Lab*

This lab has 3 Questions associated with Variograms. You need to hand in a **single PDF** produced from R Markdown, including output plots, any statistics, code, and text for answers to the questions below. When referring to the code below it may be more useful to use the .Rmd file linked above. A Ripley's K exercise using the R package, **spatstat**, is also included but is optional.

OPTIONAL: If you would like to test out GitHub in completing this lab, complete your .Rmd file in RStudio and push your PDF (and your .Rmd if you want) to a repository (if you did this in Lab 4, add this Lab 5 Markdown to the same repository). In your .Rmd file, add the link to the GitHub .Rmd file so it's visible in your PDF when you submit to D2L.

**First**, refer to the *Lab 5 handout on Variograms*

OPTIONAL RESOURCES: If you would like more information, you may also find this ArcGIS series of pages useful in explaining Semivariograms (aka variograms). Note that we won't be using ArcGIS, but the concepts are the same in R. For more of a "book" style resource, see Chapter 3: The principles of geostatistical analysis, starting on Page 49 in *Using ArcGIS Geostatistical Analyst*.

*Empirical semivariogram and covariance functions*

*Creating empirical semivariograms*

*Binning empirical semivariograms*

*Choosing a lag size*

*Empirical semivariograms for different directions*

*Semivariogram and covariance functions*

*Understanding a semivariogram: The range, sill, and nugget*

*Fitting a model to the empirical semivariogram*

*Semivariograms and Covariance Functions*

*Combining semivariogram models*

*Accounting for anisotropy using directional semivariogram and covariance functions*

**Second**, complete the *Start working in R*: section. The code below is modified from the Vignette, *Exploring spatial data in R* by Andrew Finley (MSU; <https://www.finley-lab.com>) and Sudipto Banerjee (UCLA; <http://sudipto.bol.ucla.edu/>). See the vignette for more context.

**Third**, after completing the *Start working in R*: section, answer the 3 Questions at the end of the lab.

*Start working in R*:

```
##### STARTING UP R
# Clear all existing data (or don't and just make sure you start with a newly opened RStudio session; s
rm(list=ls())

# Close graphics devices
graphics.off()

# Set the paths for your work
output_path<-"output"
# if this folder doesn't exist, create it
if(!dir.exists(output_path)){
  dir.create(output_path)
}

# Create the folders (directories) "data" and "lab5" - If they exist already, this command won't over-w
data_path<-(file.path("data","lab5"))
if(!dir.exists(data_path)){
  dir.create(data_path,recursive = TRUE)
}
```

If you previously saved your workspace and want to load it here, do so this way:

```
load(file.path(output_path,"lab5.RData"))
```

#### NOTES on R packages used in this lab:

- (1) With R Markdown, it is helpful to install packages locally before knitting (copy this into your R console before you knit). for (package in c("classInt", "spBayes", "MBA", "gstat", "fields", "rgl", "lattice")) { if (!require(package, character.only=T, quietly=T)) {

```
install.packages(package)
library(package, character.only=T)
}
}
```

We will use forest inventory data from a long-term ecological research site in western Oregon (WEF from fields package ).

```
library(classInt)
library(sf)
library(sp)
library(spBayes)
library(MBA)
library(gstat)
library(fields)
library(rgl)
library(dplyr)
library(lattice)
```

#### Spatial data visualization

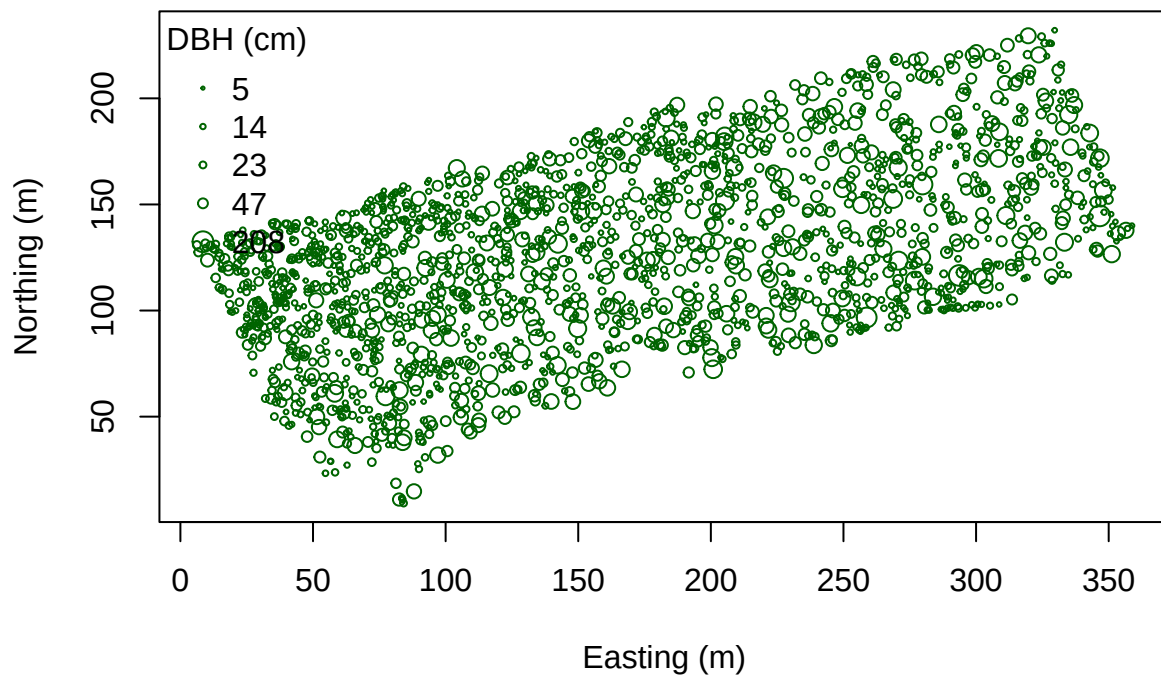
Load the data: We begin by removing rows that have NA for the variables of interest, creating a vector of DBH and HT, and plotting the tree coordinates.

```
data(WEF.dat)
?WEF.dat
WEF.dat <- WEF.dat[!apply(WEF.dat[,c("East_m", "North_m",
                                     "DBH_cm", "Tree_height_m", "ELEV_m")],
                          1, function(x)any(is.na(x))),]

DBH <- WEF.dat$DBH_cm
HT <- WEF.dat$Tree_height_m
```

Create Coordinates:

```
coords <- as.matrix(WEF.dat[,c("East_m", "North_m")])
plot(coords, pch=1, cex=sqrt(DBH)/10, col="darkgreen", xlab="Easting (m)",
      ylab="Northing (m)")
leg.vals <- round(quantile(DBH),0)
legend("topleft", pch=1, legend=leg.vals, col="darkgreen",
      pt.cex=sqrt(leg.vals)/10, bty="n", title="DBH (cm)")
```



Here we make a color ramp function then create a color palette with five shades. colorRamp and Class Intervals:

```
col.br <- colorRampPalette(c("blue", "cyan", "yellow", "red"))
col.pal <- col.br(5)

quant <- classIntervals(DBH, n=5, style="quantile")
```

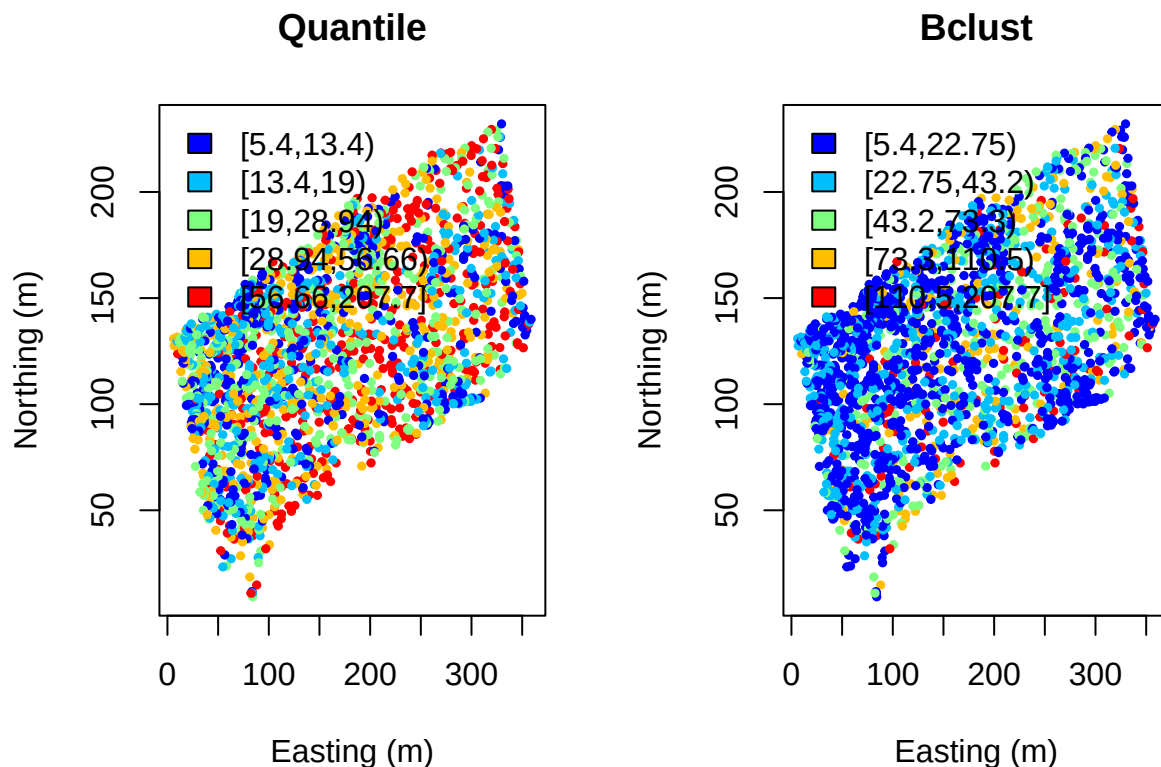
```
fisher <- classIntervals(DBH, n=5, style="fisher")
kmeans <- classIntervals(DBH, n=5, style="kmeans")
hclust <- classIntervals(DBH, n=5, style="hclust")
```

quantFisher and Plot the data:

```
quant.col <- findColours(quant, col.pal)
fisher.col <- findColours(fisher, col.pal)

par(mfrow=c(1,2))
plot(coords, col=quant.col, pch=19, cex=0.5, main="Quantile",
      xlab="Easting (m)", ylab="Northing (m)")
legend("topleft", fill=attr(quant.col, "palette"),
      legend=names(attr(quant.col, "table")), bty="n")

plot(coords, col=fisher.col, pch=19, cex=0.5, main="Bclust",
      xlab="Easting (m)", ylab="Northing (m)")
legend("topleft", fill=attr(fisher.col, "palette"),
      legend=names(attr(fisher.col, "table")), bty="n")
```

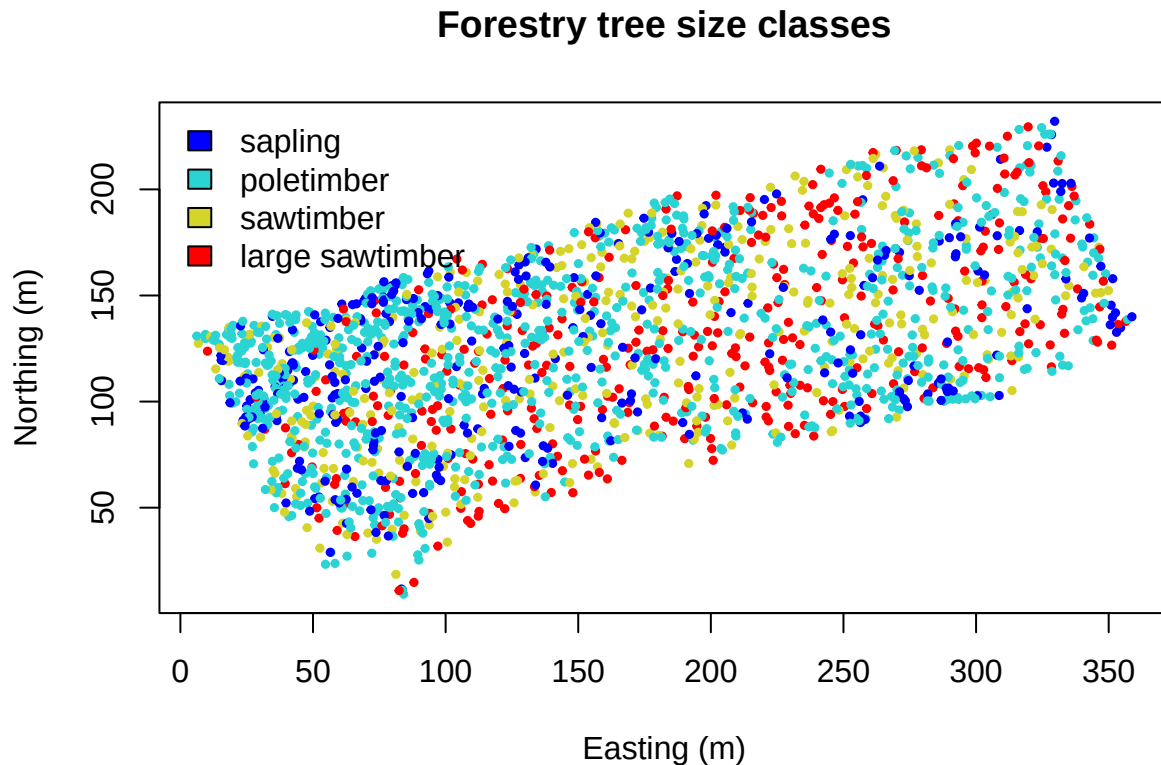


Alternatively, we can choose intervals that are meaningful to the analysis. Assign diameter at breast height (DBH) classes:

```
fixed <- classIntervals(DBH, n=4, style="fixed", fixedBreaks=c(0,12.7,30.48,60,max(DBH)+1))

fixed.col <- findColours(fixed, col.pal)

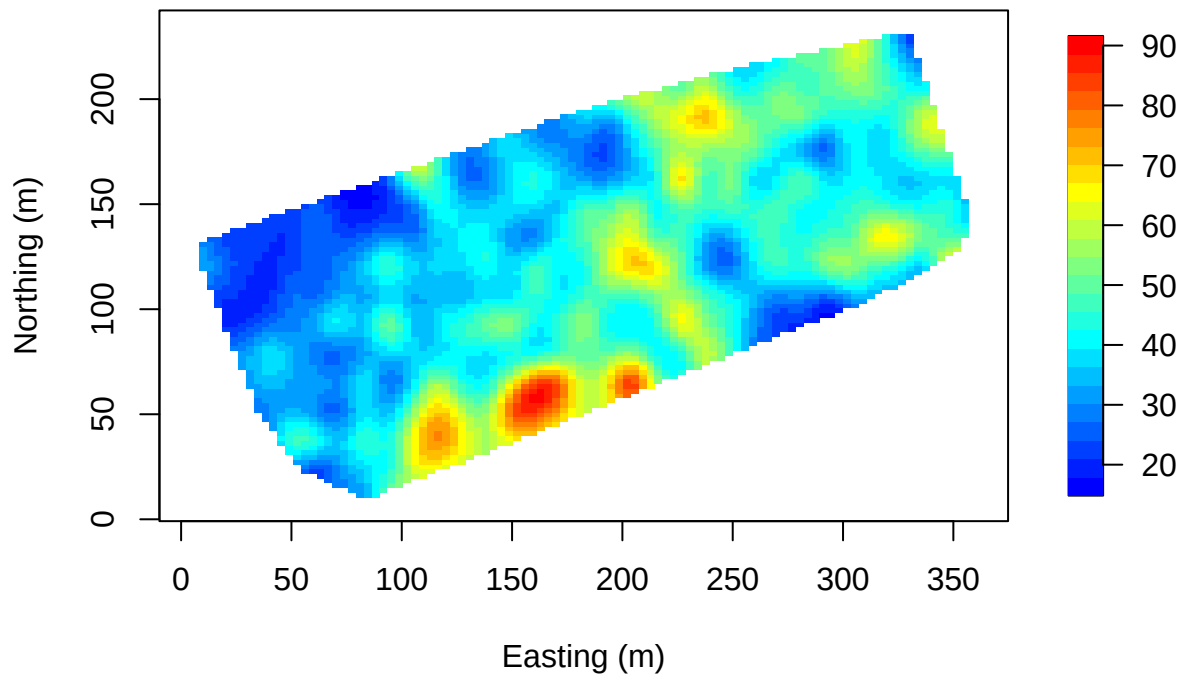
plot(coords, col=fixed.col, pch=19, cex=0.5,
      main="Forestry tree size classes", xlab="Easting (m)", ylab="Northing (m)")
legend("topleft", fill=attr(fixed.col, "palette"),
      legend=c("sapling","poletimber","sawtimber","large sawtimber"), bty="n")
```



Interpolate over the points. Note that `image.plot()` is from the “fields” R package.

```
x.res <- 100; y.res <- 100

surf <- mba.surf(cbind(coords, DBH), no.X=x.res, no.Y=y.res,
                h=5, m=2, extend=FALSE)$xyz.est
image.plot(surf, xaxs = "r", yaxs = "r",
           xlab="Easting (m)", ylab="Northing (m)", col=col.br(25))
```



Visualize in 3D - a new window opens up. Move the 3D image around with your mouse to see different perspectives!

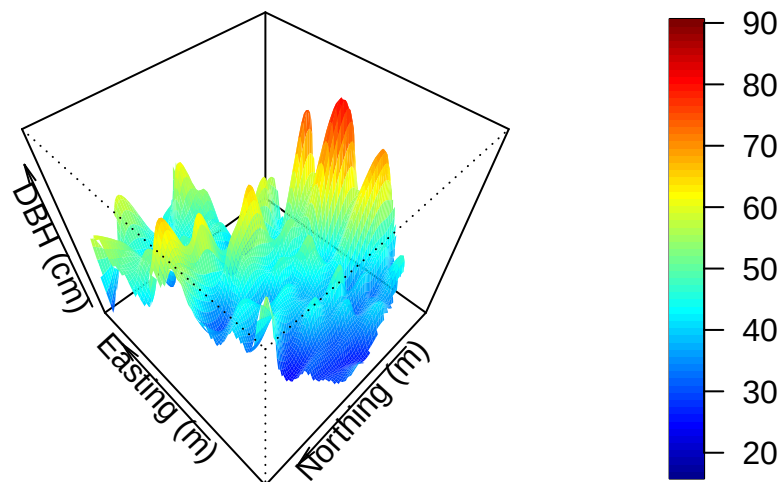
```
zlim <- range(surf[[3]], na.rm=TRUE)
zlen <- zlim[2] - zlim[1] + 1
colorlut <- col.br(zlen) # DBH color lookup table
col <- colorlut[ surf[[3]]-zlim[1]+1 ] # assign colors to heights for each point

surface3d(surf[[1]], surf[[2]], surf[[3]], col=col)
axes3d(); title3d(main="DBH", xlab="Easting (m)",
                  ylab="Northing (m)", zlab="DBH (cm)")
```

Create a perspective plot of DBH:

```
par(mfrow=c(1,1))
drape.plot(surf[[1]], surf[[2]], surf[[3]], col=col.br(150),
           theta=225, phi=50, border=FALSE,
           add.legend=FALSE, xlab="Easting (m)",
           ylab="Northing (m)", zlab="DBH (cm)")

image.plot(zlim=range(surf[[3]], na.rm=TRUE), legend.only=TRUE, horizontal=FALSE)
```



**Variogram Analysis** Fit an exponential variogram to these data and another to the residuals of a linear regression of DBH onto tree species.

```
#DBH is a dataframe. We need to transform it to a spatial object
DBH.data<-WEF.dat
DBH.data <- DBH.data %>% rename(x=East_m,y=North_m)
coordinates(DBH.data) <- ~x+y
max.dist <- 0.25*max(iDist(coordinates(DBH.data)))
bins <- 50
# variogram analysis to quantify the range of spatial dependence
vario.DBH <-variogram(DBH_cm~x+y, DBH.data,boundaries=seq(0, max.dist, length=bins))
#fit an exponential variogram model to DBH
fit.DBH<-fit.variogram(vario.DBH, model = vgm("Exp",cutoff = max.dist))
### check them out
plot(vario.DBH,fit.DBH,add=TRUE)
lm.DBH <- lm(DBH~Species, data=WEF.dat)
summary(lm.DBH)
```

```
##
## Call:
## lm(formula = DBH ~ Species, data = WEF.dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -78.423  -9.969  -3.561   10.924  118.277
##
```

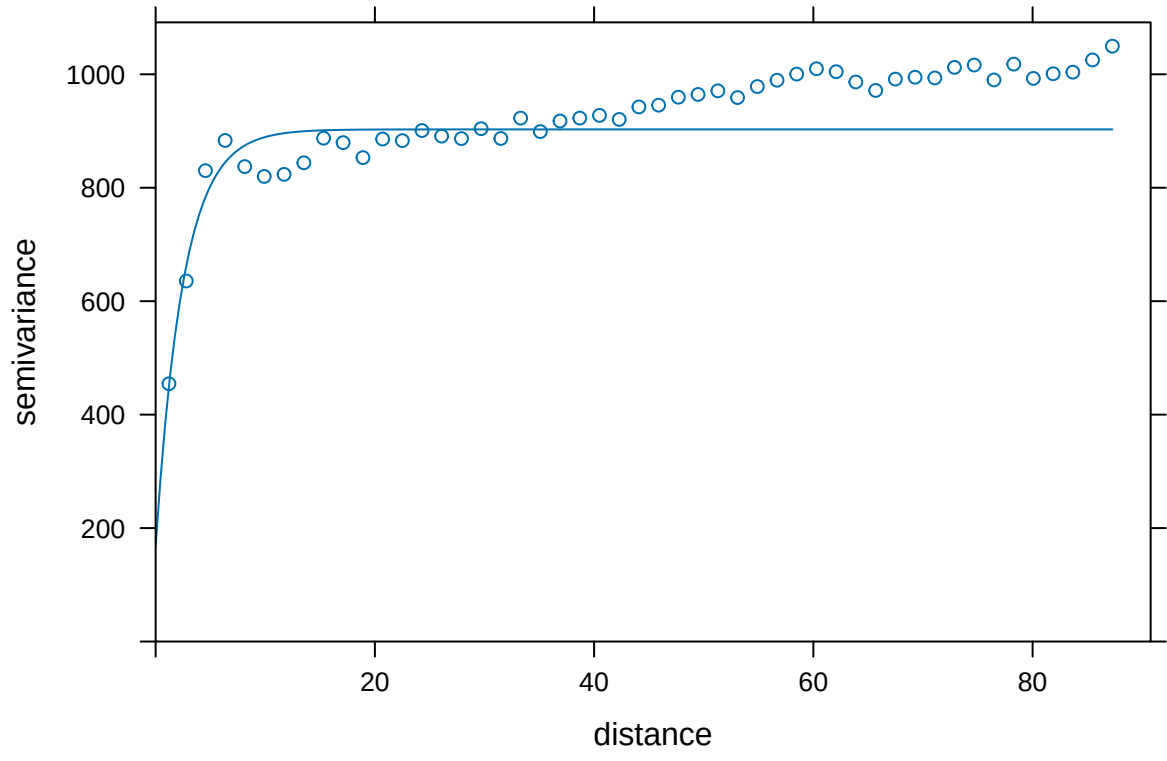
```
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)   89.423     1.303   68.629 <2e-16 ***
## SpeciesGF    -51.598     4.133  -12.483 <2e-16 ***
## SpeciesNF     -5.873    15.744   -0.373  0.709
## SpeciesSF    -68.347     1.461  -46.784 <2e-16 ***
## SpeciesWH    -48.062     1.636  -29.377 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 22.19 on 1950 degrees of freedom
## Multiple R-squared:  0.5332, Adjusted R-squared:  0.5323
## F-statistic: 556.9 on 4 and 1950 DF,  p-value: < 2.2e-16

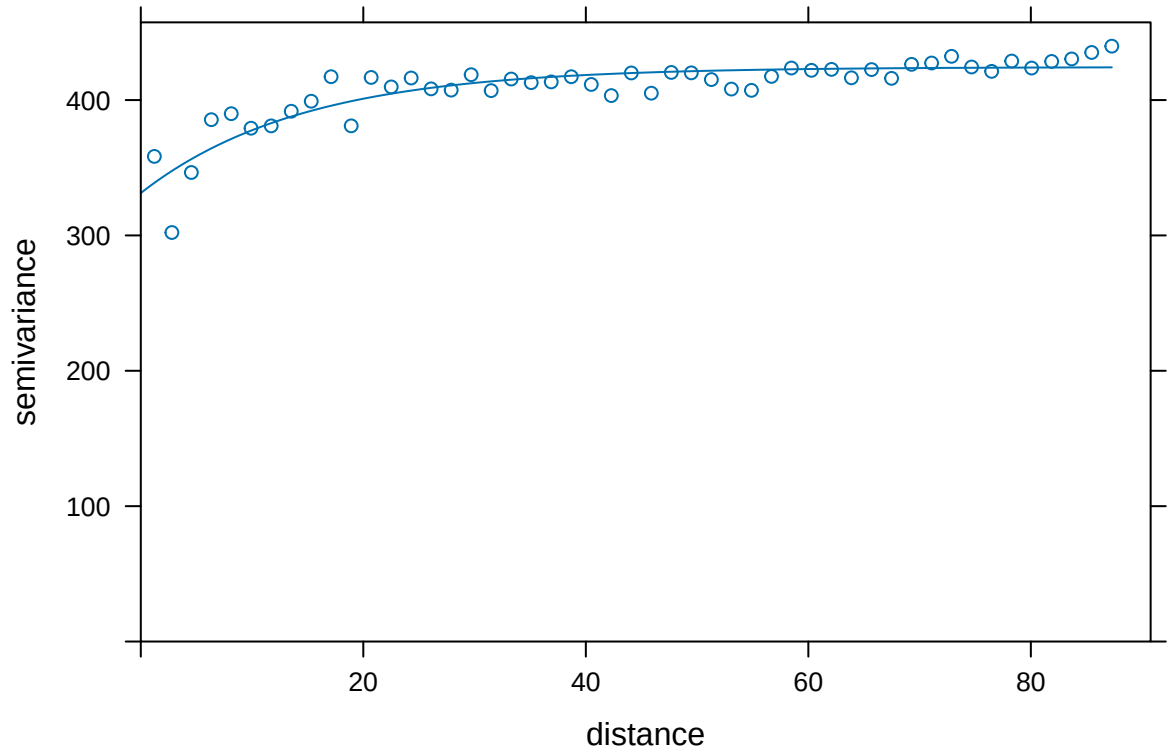
DBH.resid <- resid(lm.DBH)
vario.DBH.resid<-variogram(DBH.resid~x+y, DBH.data,boundaries=seq(0, max.dist, length=bins))
fit.DBH.resid<-fit.variogram(vario.DBH.resid, model = vgm("Exp",cutoff = max.dist))
plot(vario.DBH.resid,fit.DBH.resid)

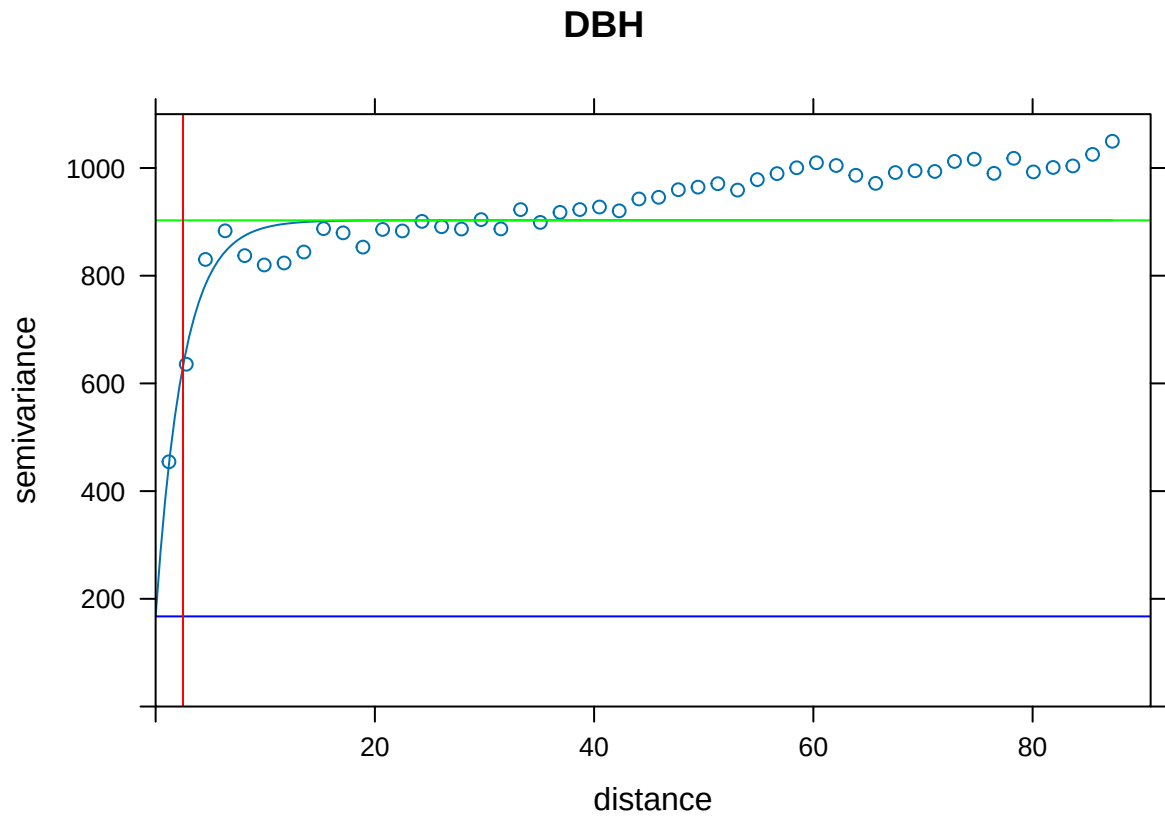
plot(vario.DBH, fit.DBH, main="DBH", ylim = c(0,1100),
      panel = function(x, ...) {
        panel = vgm.panel.xyplot(x, ...);
        panel.abline(h = c(fit.DBH$psill[1],sum(fit.DBH$psill)),col=c("blue","green"));
        panel.abline(v = fit.DBH$range[2],col="red"))}

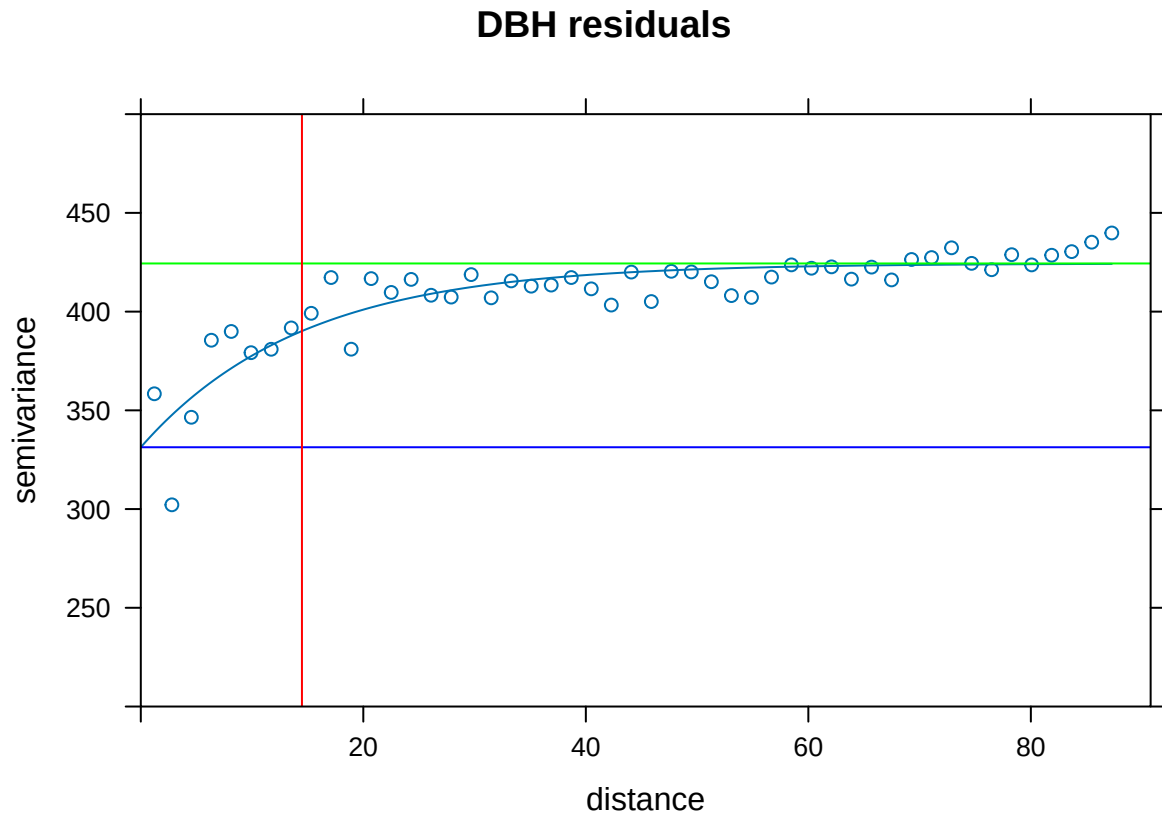
plot(vario.DBH.resid, fit.DBH.resid, main="DBH residuals", ylim=c(200,500),
      panel = function(x, ...) {
        panel = vgm.panel.xyplot(x, ...);
        panel.abline(h = c(fit.DBH.resid$psill[1],sum(fit.DBH.resid$psill)),col=c("blue","green"));
        panel.abline(v = fit.DBH.resid$range[2],col="red"))}
```





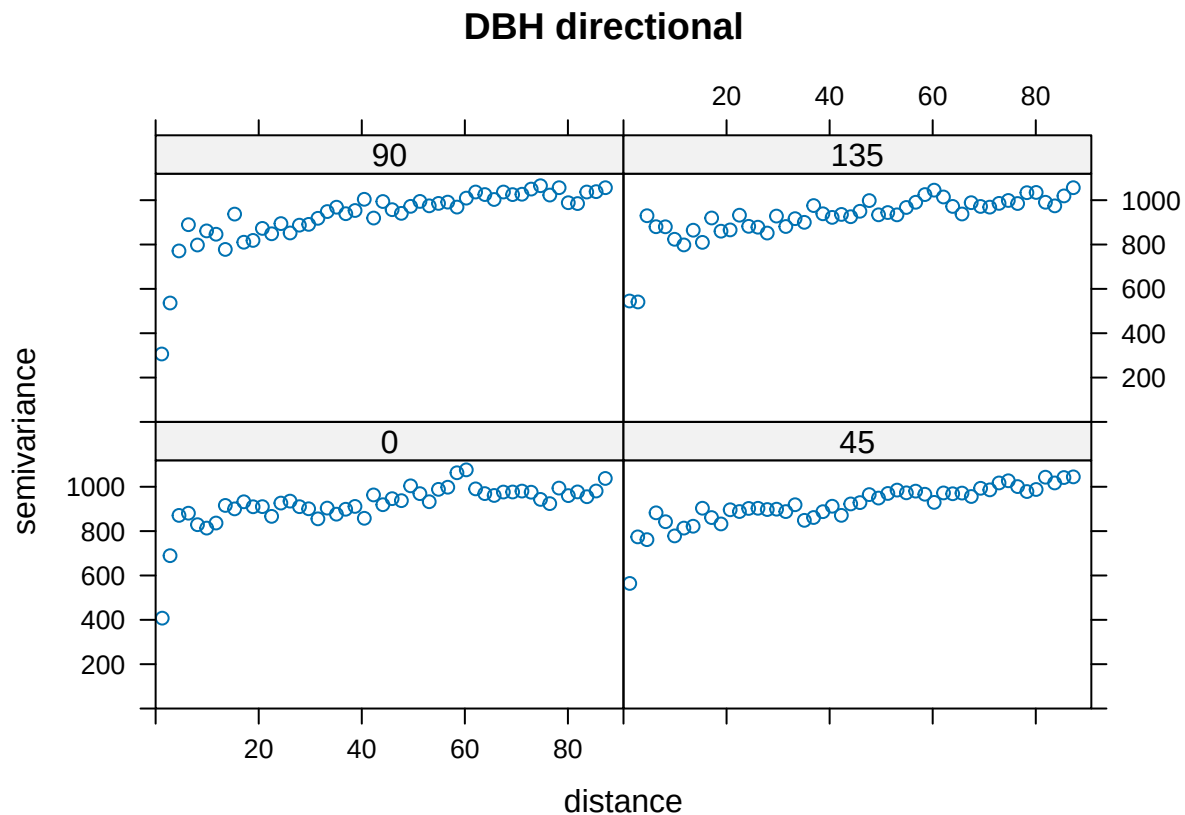






The above variograms are isotropic (spatial dependence is the same in all directions). Compute directional semivariograms to determine if there is anisotropy.

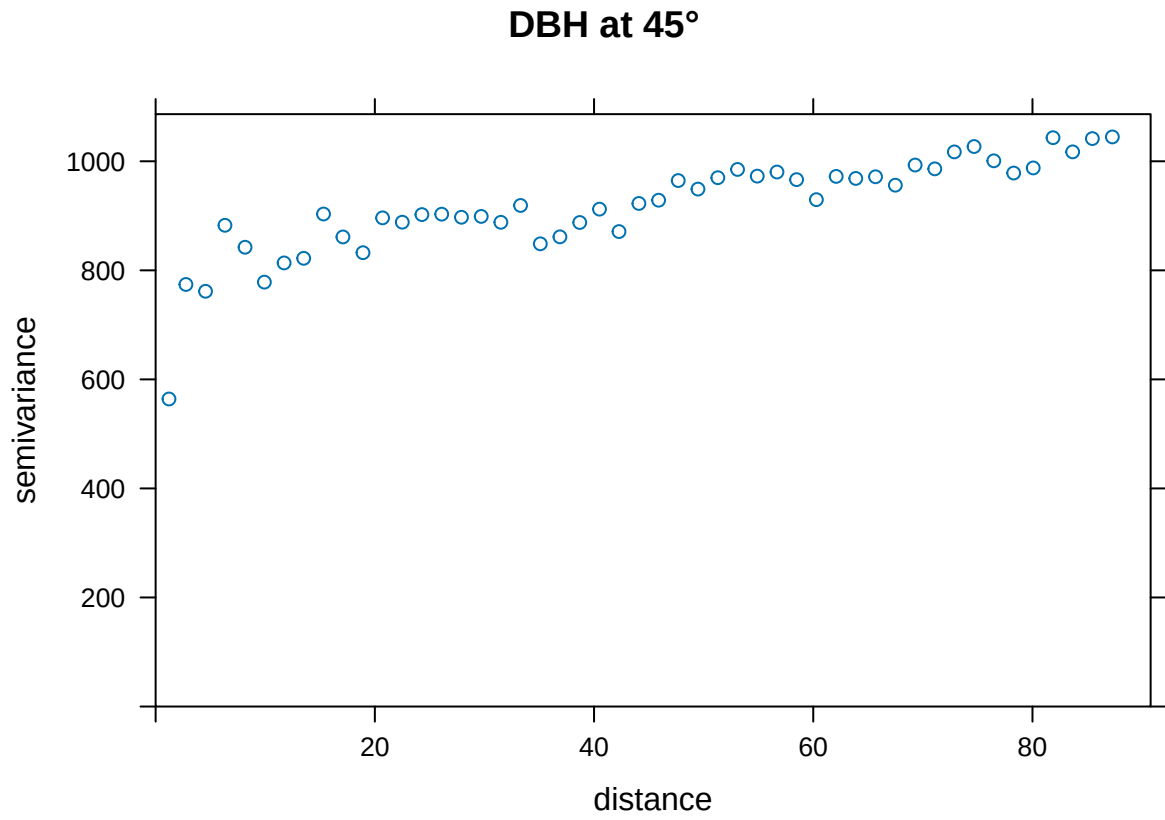
```
directional.vario.DBH <- variogram(DBH_cm~x+y,DBH.data,boundaries=seq(0, max.dist, length=bins),alpha=c
par(mfrow=c(1,1))
plot(directional.vario.DBH, main="DBH directional", lty=1)
```

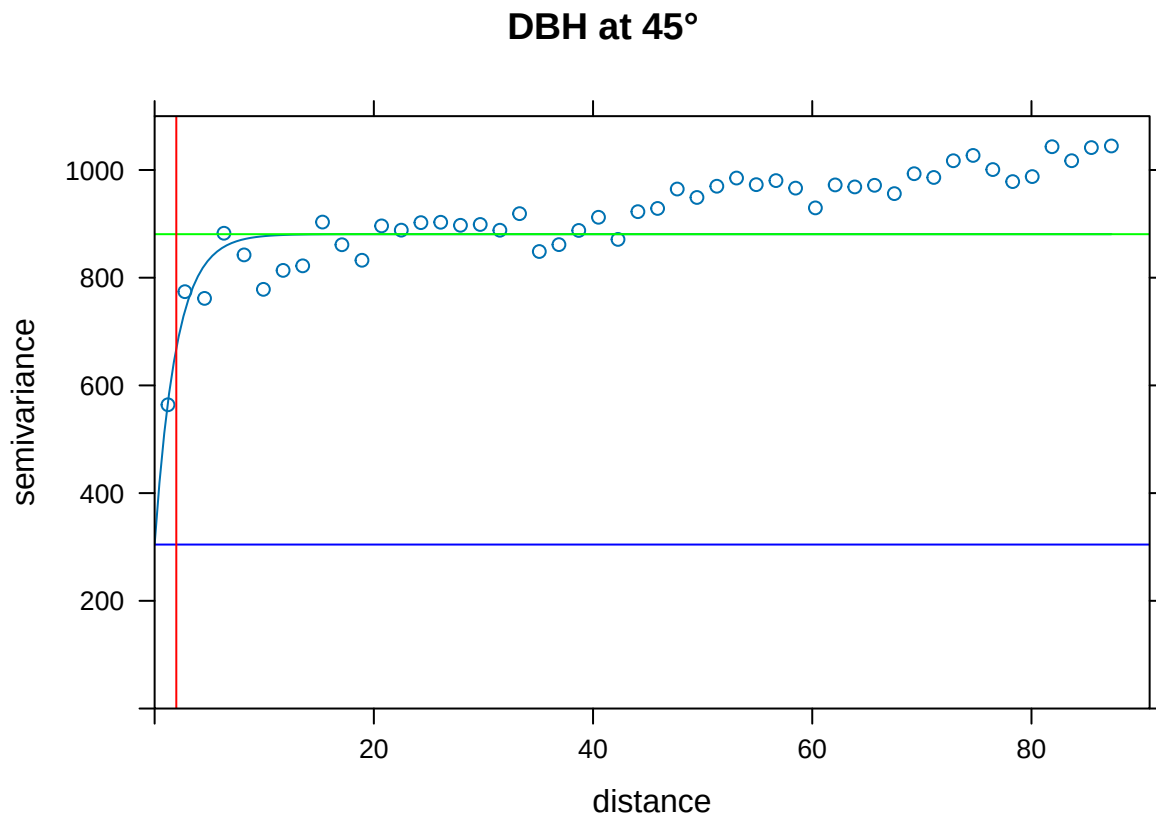


**End of Exploring spatial data in R** Want to fit a variogram function for just one direction? In this case, shown for 45°.

```
par(mfrow=c(1,1))
plot(directional.vario.DBH[directional.vario.DBH$dir.hor==45,], main="DBH at 45°", lty=1)
vario.DBH.45<-directional.vario.DBH[directional.vario.DBH$dir.hor==45,]
# Apply fit
fit.DBH.45<-fit.variogram(vario.DBH.45, model = vgm("Exp",cutoff = max.dist))

# Plot it
plot(vario.DBH.45, fit.DBH.45, main="DBH at 45°", ylim = c(0,1100),
      panel = function(x, ...) {
        panel = vgm.panel.xyplot(x, ...);
        panel.abline(h = c(fit.DBH.45$psill[1],sum(fit.DBH.45$psill)),col=c("blue","green"));
        panel.abline(v = fit.DBH.45$range[2],col="red")})
```





### QUESTION 1:

Create a directional variogram for 4 directions OTHER than the ones used in the example (i.e., other than 0, 45, 90, 135). Fit the variogram for each direction. Show your code and resulting plot. Explain why you chose these other 4 directions. Does it suggest these data are anisotropic? Why or why not?

### QUESTION 2:

What do the variograms produced in question 1 tell you about these data? Specifically discuss the sill, nugget, and range.

### QUESTION 3:

Why would you want to produce a variogram on the residuals? What does a variogram using residuals tell you that's different from a variogram on the original data?

## Ripley's K

This is an OPTIONAL exercise.

The following text is modified from the `spatstat` package: What is Ripley's K? It's the K function (or the "reduced second moment function") of a stationary point process  $X$ , and is defined so that  $\lambda K(r)$  equals the expected number of additional random points within a distance  $r$  of a typical random point of

X. Lambda is the intensity of the process, i.e., the expected number of points of X per unit area. The K function is determined by the second order moment properties of X.

In other words, Ripley's K summarizes spatial dependence in the form of clustering or dispersion over a specified range of distances (scales).

An estimate of K derived from a spatial point pattern dataset can be used in exploratory data analysis and formal inference about the pattern (Cressie, 1991; Diggle, 1983; Ripley, 1977, 1988). In exploratory analyses, the estimate of K can indicate inter-point "dependence" and "clustering". For inferential purposes, the estimate of K is usually compared to the true value of K for a completely random (Poisson) point process (Null expectation), which is  $K(r) = \pi * r^2$ . Deviations between the empirical and theoretical K curves may suggest spatial clustering or spatial regularity.

In the **spatstat** package, **Kest** estimates the K function of a stationary point process, given observation of the process inside a known, bounded window. The argument X is interpreted as a point pattern object (of class "ppp", see **ppp.object**) and can be supplied in any of the formats recognized by **as.ppp()**.

The ArcGIS page on this topic is [Multi-Distance Spatial Cluster Analysis \(Ripley's K Function\) \(Spatial Statistics\)](#)

You can visualize a measure of spatial clustering/dispersion over a range of distances as in this ArcGIS schematic:

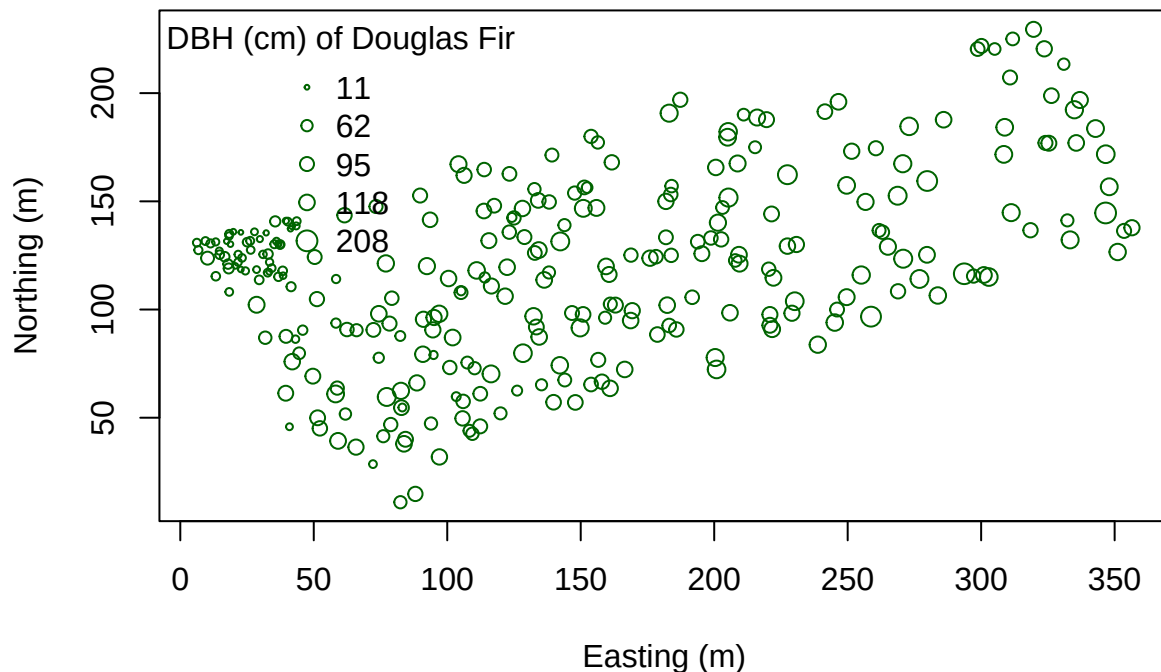
Suppose we want to determine how Douglas Fir trees are distributed in these data - are they Clustered? Dispersed? Random? At what scale? And is the pattern significant? First pull out Douglas Fir from the dataset. Then plot it.

```
library(spatstat)

dfir<-subset(WEF.dat, Species=="DF")

# Plot Douglas Fir DBH distribution
dfDBH <- dfir$DBH_cm
coords.df <- as.matrix(dfir[,c("East_m","North_m")])
plot(coords.df, pch=1, cex=sqrt(dfDBH)/10, col="darkgreen", xlab="Easting (m)",
      ylab="Northing (m)")
leg.vals <- round(quantile(dfDBH),0)
legend("topleft", pch=1, legend=leg.vals, col="darkgreen",
      pt.cex=sqrt(leg.vals)/10, bty="n", title="DBH (cm) of Douglas Fir")
```



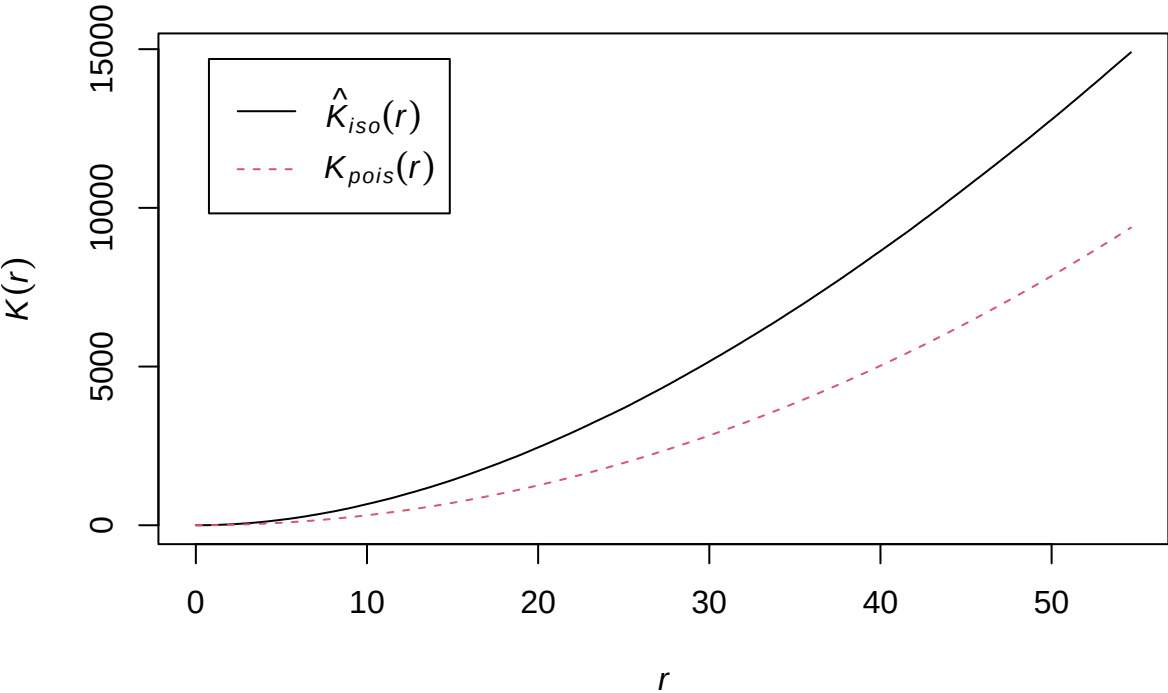


You can start to see some clustering of small DBH in the northwestern corner. Let's quantify the distribution across the study area. Create a `SpatialPointsDataFrame` to generate a point pattern object from the Douglas Fir data. Then apply an edge correction. The edge correction is necessary because a hard edge (boundary) on the data extent will under-estimate the pattern since there are no neighboring points outside the data boundary. We need to specify which points will be used for the correction vs. the actual calculation. See this diagram in [Multi-Distance Spatial Cluster Analysis \(Ripley's K Function\) \(Spatial Statistics\)](#).

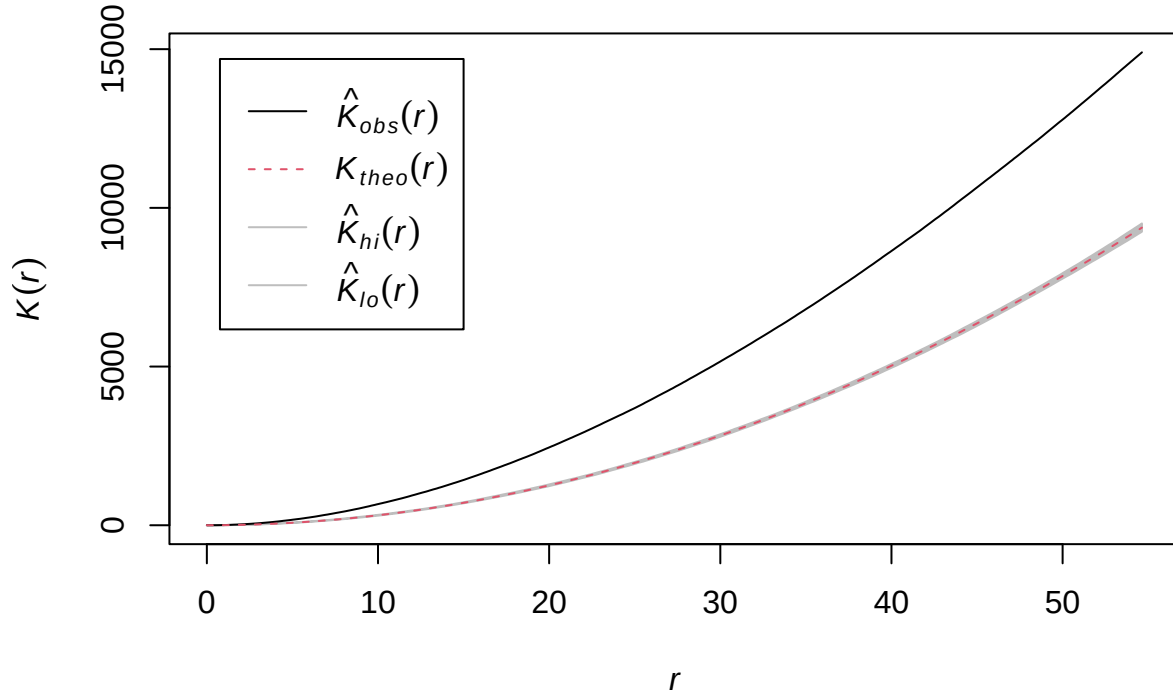
We will use the `best` option for the correction from the `Kest` function:

```
dflr.sp<-dflr
coordinates(dflr.sp) <- c("East_m","North_m")
# Create the point pattern object
#Make an owin (observation window)
bb <- bbox(dflr.sp)
win <- owin(xrange = bb[1, ], yrange = bb[2, ])
dflr.ppp <- ppp(x = coords[,1], y = coords[,2], window = win)
# Apply the Kest function, selecting the "best" option for edge correction.
K.dflr <- Kest(dflr.ppp, correction="best")
plot(K.dflr, main="Ripley's K function for Douglas Fir")
# Is the pattern significant? Apply an envelope using 500 simulations to create the 95% confidence interval
plot(envelope(dflr.ppp, Kest, correction="best", nsim = 500))
```

Ripley's K function for Douglas Fir



**envelope(dfir.ppp, Kest, correction = "best", nsim = 500)**



Notice how the entire plot for Douglas Fir data (black solid line) is above the theoretical curve. Values higher than the theoretical line (red dotted line) reflect clustering, and values below the theoretical line reflect dispersion. The plot indicates that even though we observed clustering in one region by plotting the occurrence data above, Douglas Fir is clustered across the entire study area, at nearly all scales. At very local scales (close to  $r=0$ ) the curves overlap indicating that there is not strong clustering at those short distances. Repeat the process above to try it out on another species to see how two species compare.

This work is licensed under a Licensed under CC-BY 4.0 2025 by Phoebe Zarnetske.